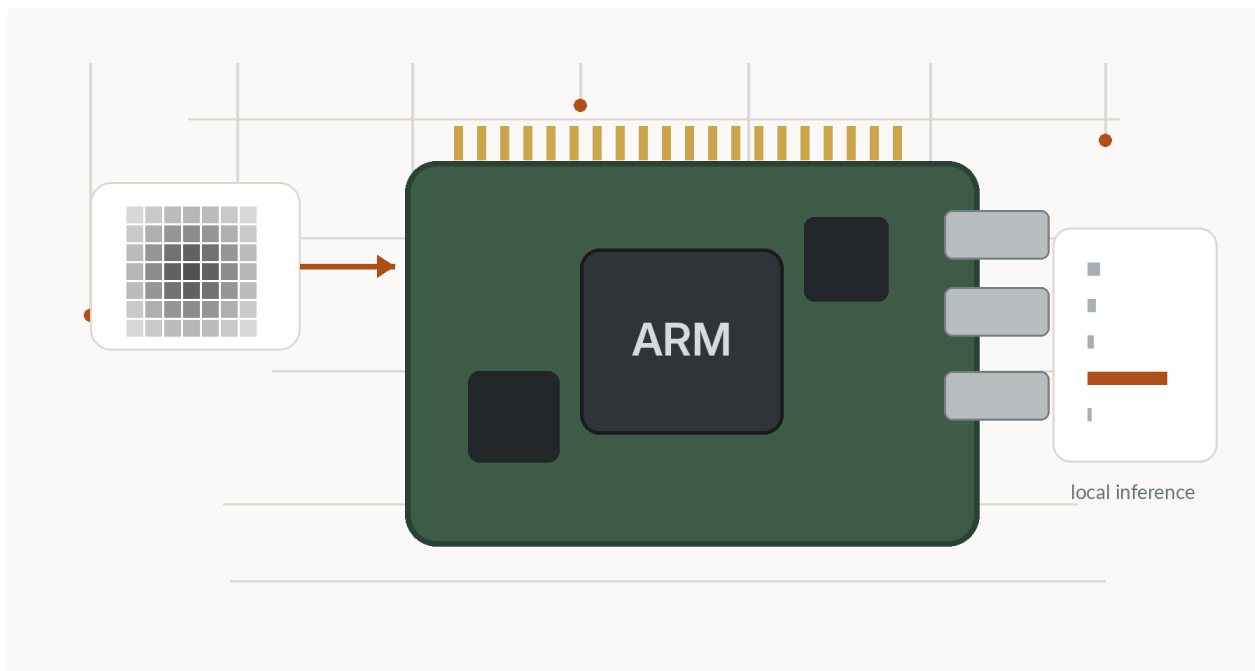


POSTGRADUATE PROGRAMME IN EMBEDDED SYSTEMS

Artificial Intelligence in Embedded Systems

PRACTICAL LABORATORY 02

From training to deployment: build, convert and run your own model



19 September 2026 | Tiago Fonseca | ISEP | v6 FINAL

LAB PURPOSE

Train and compare a baseline and an MLP, investigate one controlled change, convert the reference MLP into an inference artifact, and verify that the same artifact runs correctly on the Raspberry Pi.

1.1 Mission of PL2

In PL1 you received a trained model and learned how to inspect its data contract and execute inference on the Raspberry Pi. In PL2 the direction reverses: you will create the model yourself, evaluate whether the extra complexity is justified, convert the trained model into a deployment artifact and prove that the same artifact runs correctly on the Raspberry Pi.

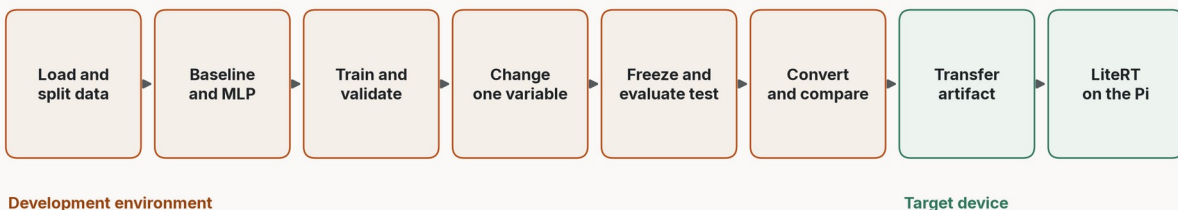
CORE ENGINEERING QUESTION What evidence is required before we can claim that a model trained in a notebook has become a deployable embedded-AI component?

1.2 Where PL2 sits in the practical sequence

Session	Main responsibility
PL1 · Observe and execute	Understand the device, dataset, tensor contract and inference.
PL2 · Build and deploy	Train a baseline and MLP, evaluate them, convert to .tflite and deploy.
PL3 · Optimise	Study representation, Float32/Float16/INT8 and quantisation effects.
PL4 · Decide	Design an experiment and defend a deployment decision with evidence.

PL2 · From training to deployment

Develop on the computer; collect deployment evidence on the Raspberry Pi.



PL2 workflow: training evidence is produced off-device; deployment evidence is collected on the Raspberry Pi.

1.3 Working method

Use the official PL2 v6 notebook in Google Colab for training and conversion. Then validate deployment on the Raspberry Pi. Keep these responsibilities separate throughout the laboratory.

WORK THROUGH ONE CELL AT A TIME The supplied notebook already contains the code. Read and run one cell at a time, wait for it to finish, inspect its output, and then continue. Use the shaded boxes in this PDF to follow the explanation. Do not paste duplicate code into the notebook.

DO NOT INSTALL TENSORFLOW ON THE RASPBERRY PI The Pi remains an inference target using the LiteRT environment prepared in PL1. Full TensorFlow/Keras is used in Colab for training and conversion.

2

Background for PL2

The minimum theory needed to understand training, comparison and deployment.

2.1 Training is not inference

Inference applies fixed learned parameters to a new input. Training changes those parameters by repeatedly comparing predictions with known labels, computing a loss, and using an optimiser to reduce that loss. The computational cost and software stack required for training are therefore much larger than those required for inference.

Training	Inference
Uses labelled examples and updates weights.	Uses fixed weights to produce outputs.
Requires forward pass + loss + backpropagation.	Requires only the forward computation.
Usually performed on workstation/cloud/GPU.	In this UC, validated on Raspberry Pi CPU.
Produces a model artifact.	Consumes that artifact at runtime.

A useful way to separate the two phases is to ask whether model parameters are allowed to change. During training, weights and biases are mutable: a batch is processed, a prediction is produced, a loss quantifies the error, gradients describe how that loss changes with each parameter, and the optimiser updates the parameters. During inference, the learned parameters are frozen; the system performs only the forward computation required to produce outputs.

The same neural network can therefore have very different computational and software requirements during training and deployment. Training needs gradient computation, optimizer state and repeated passes over many samples; inference needs only the operations and parameters required for the forward pass. This distinction is why PL2 trains in Colab but validates deployment on the Raspberry Pi.

2.2 Train, validation and test: three different jobs

The training set is used to update the model parameters. Validation data is used during development to observe generalisation and guide decisions such as capacity or number of epochs. The test set is kept outside the training process and is used only for final evaluation. Reusing the test set to tune the model leaks information and makes the final performance estimate optimistic.

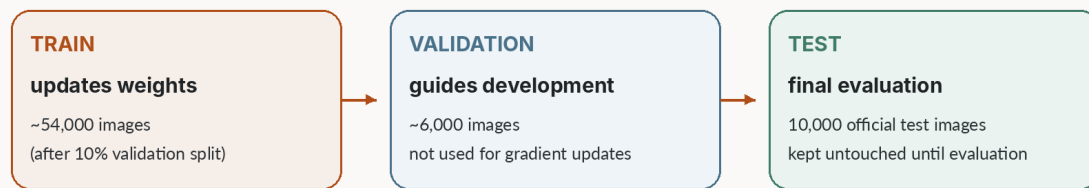
POSTGRADUATE-LEVEL QUESTION If two models are compared on the same test set but one was repeatedly tuned after looking at test errors, is the comparison still methodologically clean? Explain what information has leaked.

We reserve 6,000 shuffled examples from the official 60,000-image training split for validation and train on the remaining 54,000. The same split is reused for every model. The 10,000 official test examples remain separate until development decisions are frozen. Keras `validation_split` reserves the last samples before shuffling; an explicit split makes our protocol visible.

This separation is methodological, not administrative. If you repeatedly inspect the test result and change the architecture, preprocessing or number of epochs because of what you saw, the test set has become part of development and no longer represents an independent final check.

Three datasets, three different responsibilities

PL2 keeps the official test set isolated while training decisions are made with training and validation evidence.



Key rule: if you repeatedly inspect the test result and change the model because of it, the test set has effectively become part of development.

Training updates parameters; validation guides development; test evidence should remain independent of model tuning.

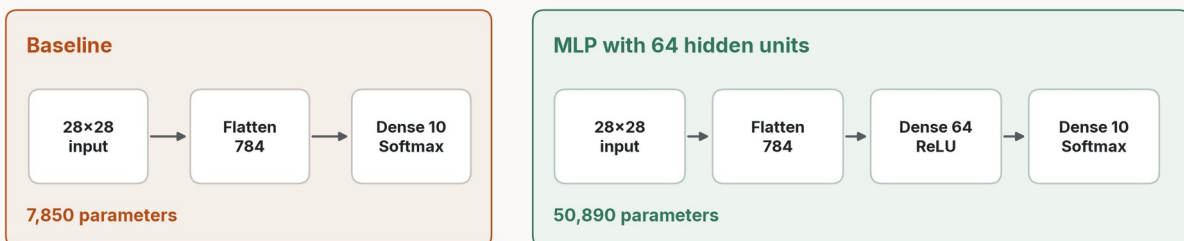
2.3 Why start with a baseline?

A baseline is deliberately simple. It gives a reference point against which additional model capacity must justify itself. Without a baseline, a more complex network may look impressive without showing that the complexity actually buys useful predictive performance.

The PL2 baseline is equivalent to multinomial logistic regression on the 784 image pixels: Flatten → Dense(10, Softmax). The MLP adds one hidden Dense layer with 64 ReLU units. Both receive exactly the same normalized input and predict the same 10 classes.

Two models with the same input and output

Hold data and training settings fixed; compare architecture and validation results.



Controlled comparison: identical data and training budget; the architecture changes.

PARAMETER CALCULATION Baseline: $(784 + 1) \times 10 = 7,850$ trainable parameters. MLP: $(784 + 1) \times 64 + (64 + 1) \times 10 = 50,890$. Bias terms matter.

Both models use the same split, normalization, optimizer, learning rate, loss, batch size and five-epoch training budget. We reset the seed before each model. The architecture changes: the MLP adds both parameters and a non-linear hidden layer. Equal epochs control the number of passes through the data, not the computation time. One seeded run supports a classroom comparison, not a universal ranking.

2.4 Dense layers, ReLU and Softmax

A Dense layer computes weighted combinations of all inputs plus a bias. ReLU introduces a non-linear transformation, allowing the hidden layer to represent structure that a single linear decision surface cannot. The final Softmax converts the 10 output activations into non-negative scores that sum approximately to one.

Softmax scores are useful ranking signals, but they are not automatically calibrated probabilities of being correct. PL1 already showed that a model can be highly confident and wrong.

For one Dense layer, each output unit computes a weighted sum of its inputs plus a bias. In matrix form this is often written as $z = Wx + b$. The learnable values in W and b are the parameters adjusted during training. A layer with 784 inputs and 64 outputs therefore contains 784×64 weights plus 64 biases.

ReLU applies $\max(0, z)$ element-by-element. Its importance is not that it “classifies” anything by itself, but that it introduces non-linearity; without a non-linear activation between Dense layers, stacking linear layers would still collapse to a single linear transformation. Softmax then turns the final 10 logits into non-negative scores that sum to one, making the classes easy to rank. Those scores are not automatically calibrated probabilities of correctness.

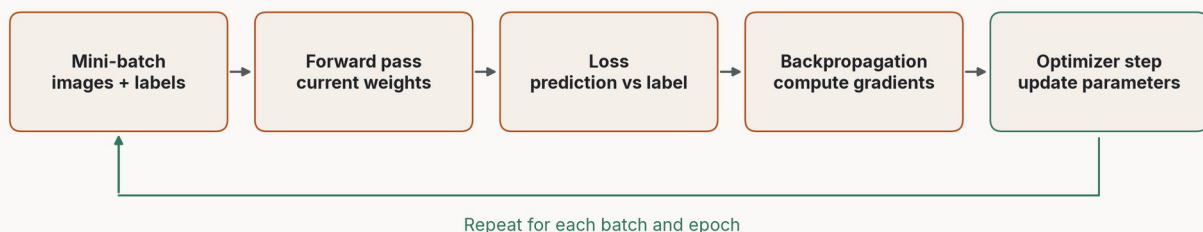
2.5 How the network learns: forward pass, loss and backpropagation

Training alternates between prediction and correction. A mini-batch passes through the current network (forward pass), the loss compares outputs with the known labels, backpropagation applies the chain rule to compute gradients, and the optimiser uses those gradients to update the parameters. The cycle then repeats for the next batch.

Backpropagation is therefore not a second model and not a deployment operation. It is the mechanism used during training to assign credit or blame to parameters. When the model later runs on the Raspberry Pi, this update path disappears: only the forward inference graph is required.

What training actually does

The forward pass is also used for inference; gradient computation and updates are training work.



Conceptual training loop. Only the forward-pass portion is needed during inference.

2.6 Loss, optimiser, epochs and batches

Term	Meaning in this laboratory
Sparse categorical cross-entropy	Penalises the model when probability mass is assigned away from the correct integer class.
Adam optimiser	Uses gradients of the loss to update weights. The default learning rate is sufficient for this controlled exercise.
Batch	A subset of training examples used for one parameter update.
Epoch	One pass through the training set.
Validation accuracy/loss	Evidence about behaviour on data not used for that update.

2.7 Parameter count and model capacity

Parameter count gives a first-order view of model capacity and storage cost. For the baseline, the Dense(10) layer receives 784 inputs: 784×10 weights + 10 biases = 7,850 trainable parameters. For the MLP, Dense(64) contributes $784 \times 64 + 64 = 50,240$ parameters and Dense(10) contributes $64 \times 10 + 10 = 650$, for a total of 50,890.

More parameters allow the model to represent richer decision boundaries, but capacity is not free. It increases the amount of learned state, normally increases computation, and can make memorisation easier. Parameter count is therefore evidence about complexity, not proof of better accuracy, latency or generalisation.

2.8 Generalisation, underfitting and overfitting

More parameters increase representational capacity but do not guarantee better generalisation. A useful model must improve on unseen data, not merely fit the training set. A widening gap between training and validation performance can be evidence of overfitting, although the interpretation always depends on the learning curves and the data.

Underfitting occurs when the model or optimisation process is too limited to capture useful structure: both training and validation performance remain weak. Overfitting is different: training performance continues to improve while validation performance stops improving or deteriorates. A small train-validation gap is not a guarantee of good generalisation, but a widening gap is a useful warning signal.

Learning curves provide temporal evidence that a final accuracy value hides. In PL2, examine both accuracy and loss. Loss is especially useful because it can worsen before accuracy visibly changes, revealing that the model is becoming more confident in a way that does not generalise.



Schematic learning curves. The exact shape varies by run; interpret trends rather than expecting these curves literally.

2.9 Reproducibility and stochastic training

Neural-network training is stochastic. Random initialization, mini-batch ordering and some accelerator operations can produce slightly different parameter values and metrics between runs. Setting random seeds reduces unnecessary variation and makes a classroom comparison easier to interpret, but a seed does not turn one run into universal evidence.

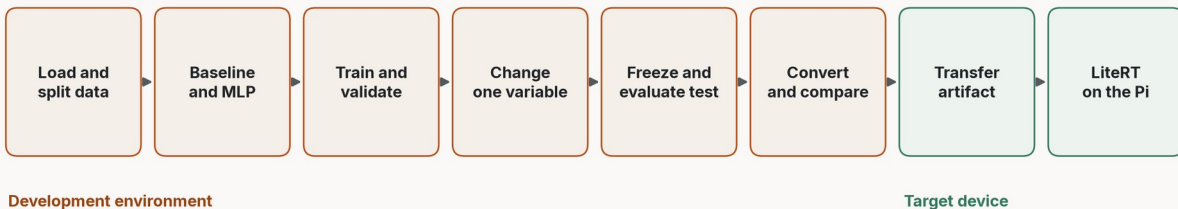
Record Python, TensorFlow, Keras, NumPy and LiteRT versions, the data split, architecture, preprocessing, epochs, batch size and seed. Reproducibility means another person can reconstruct the conditions and obtain comparable results; floating-point outputs need not be bit-for-bit identical across platforms.

2.10 From TensorFlow/Keras to LiteRT

TensorFlow/Keras is used here as the training framework: it builds the network, stores trainable variables, computes gradients and performs optimisation. After training, the model is converted to the .tflite format. That serialized artifact contains the inference graph and learned parameters required by the lightweight runtime. The Raspberry Pi then loads the artifact with LiteRT; it does not need the training machinery.

PL2 · From training to deployment

Develop on the computer; collect deployment evidence on the Raspberry Pi.



The training framework and the deployment runtime have different responsibilities.

IMPORTANT BOUNDARY Conversion is not a guarantee of correct deployment. You must still verify input shape, dtype, preprocessing, outputs and behaviour on the target device.

The saved Keras model and the .tflite artifact serve different purposes. The Keras file preserves a development/training representation suitable for continued work in TensorFlow. The .tflite file is a serialized inference artifact intended for a lightweight runtime. Conversion translates the graph and parameters into that deployment format; it does not automatically validate preprocessing, semantic labels, robustness or target-device performance.

The deployment boundary

Training produces parameters. Deployment turns those parameters into an executable contract on a target system.



PL1 starts at the export/deploy boundary: the model is already trained, but you must recover and respect the assumptions that make inference meaningful.

MODEL FILE ≠ COMPLETE SYSTEM Preprocessing, runtime, hardware, labels, thresholds and provenance are part of the deployed behaviour.

Deployment boundary: training produces an artifact; the target runtime must still satisfy the same data and tensor contract.

2.11 What remains constant from PL1

- Fashion-MNIST still uses 28×28 grayscale images and the same 10 class labels.
- The model input is still expected to be float32 values normalized from 0..255 to 0..1.
- The final model must still execute locally through LiteRT on the Raspberry Pi.
- The difference is ownership: in PL2 the model artifact is produced by your group, not supplied by the instructor.

3

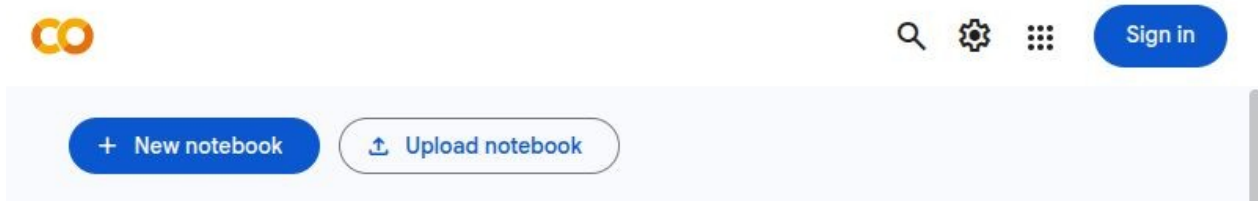
Practical laboratory

Build, compare, convert and verify a complete deployment pipeline.

3.0 Set up Google Colab on your computer

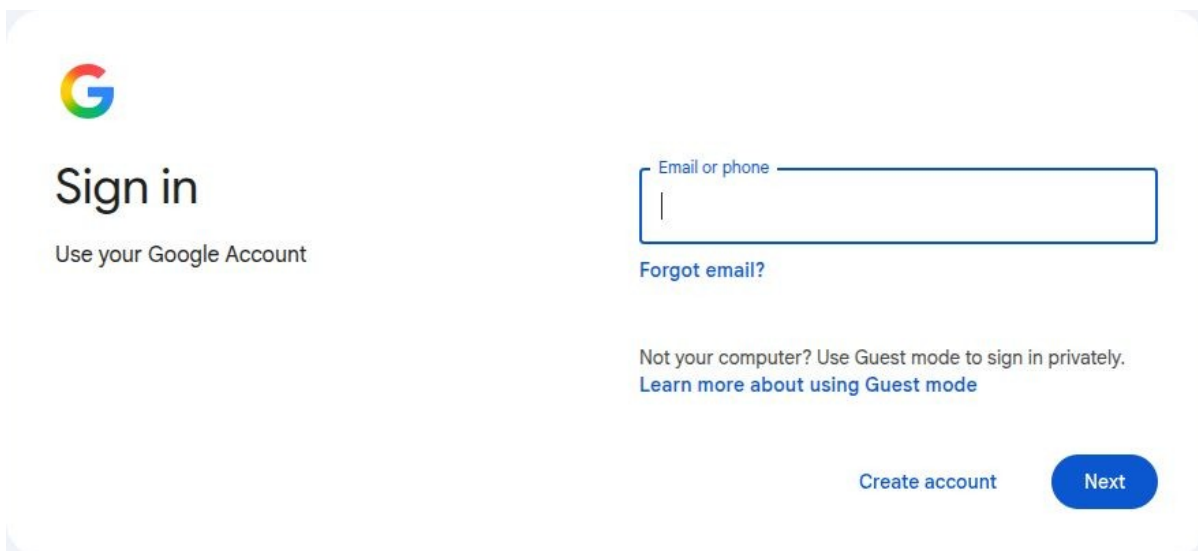
Google Colab combines explanatory text and executable Python cells in your browser. Training runs on a Google-hosted computer. Use your laptop or desktop for this setup; the Raspberry Pi is used later for inference and memory measurements.

1. Open <https://colab.research.google.com/> and click Sign in at the top right. Use a Google account that you can access during the laboratory.



Colab home: Sign in, New notebook and Upload notebook. Interface labels may vary with language.

2. Enter your Google email address, click Next, and complete the password and any account verification requested by Google. Return to Colab after signing in. If your institution blocks Colab, contact the lecturer.



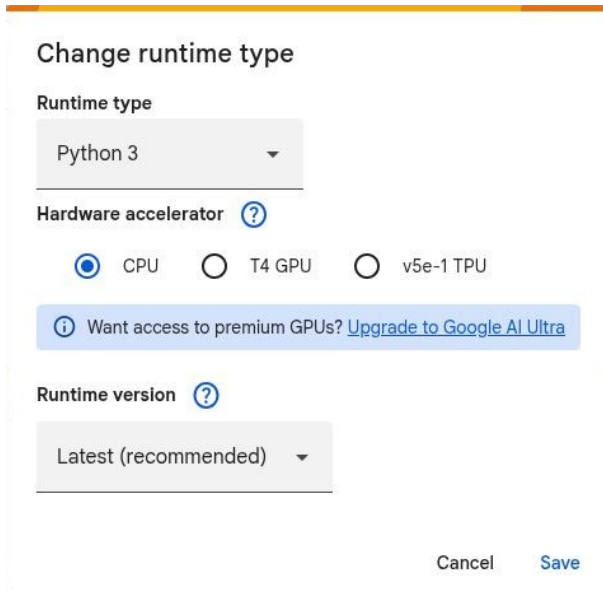
The Google sign-in page. Enter your own account details in the browser.

3. Open the official PL2 v6 notebook linked in 3.1. Choose File > Save a copy in Drive and rename your copy PL2_GroupXX.ipynb. New notebook creates a blank file; use that option only if the lecturer asks you to.

IMPORTANT · USE THE PROVIDED v6 NOTEBOOK Use the link in 3.1 and work in your own Drive copy. If the lecturer supplies the matching v6 .ipynb file instead, use Upload notebook. The code and explanations are already included; do not paste duplicate blocks.

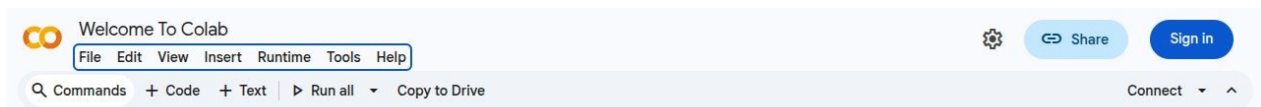
3.0 Connect the runtime and run your first cell

4. Open Runtime > Change runtime type. Select Python 3 and CPU, then Save. Use the standard runtime; these small networks do not require a paid GPU.



Runtime settings: Python 3 and CPU. This screenshot shows the public Colab welcome notebook.

5. Click Connect near the top right and wait for the runtime to become available. The notebook stores your text and code; the runtime holds the live variables and temporary files.



Use Code or Text to add cells, Copy to Drive to make a copy, and Connect to start a runtime.

6. In your supplied notebook copy, run the setup-check cell below with its play button or Shift+Enter. You should see PL2 ready and 784. Code and Text add new cells; the laboratory cells are already provided.

COLAB · Check that a code cell executes

```
print("PL2 ready")
print(28 * 28)
```

7. Execute cells from top to bottom and wait for each to finish. A spinning indicator means the cell is busy; the output appears below it. If a cell raises an error, read the last error line before continuing. A NameError often means that an earlier cell has not run in this runtime.

SAVE YOUR WORK Your notebook copy is saved in Drive. Keep your observations in text cells and save a local .ipynb copy through File > Download when required. Download model files and ZIPs separately: saving the notebook does not preserve every file in the runtime. After a restart or a new session, rerun the setup and preceding cells.

3.1 Prepare the training notebook

GOOGLE COLAB ONLY

NOT ON THE RASPBERRY PI

Run training, evaluation, conversion and downloads in Google Colab on your computer. The Raspberry Pi work begins at 3.10B, after you return from the notebook.

IMPORTANT — Use only the official PL2 v6 notebook provided with this handout.

[Open the official PL2 v6 notebook in Google Colab](#)

File > Save a copy in Drive. Rename your copy PL2_GroupXX.ipynb.

Work in your own Colab copy now

Follow the notebook from top to bottom: setup, training, validation, conversion and downloads. Read the explanations and record your observations there. The optional Extra is already placed before the final test evaluation; skip it only if instructed.

The following PDF pages reproduce the notebook for reference. They are not an additional set of tasks to execute or paste into Colab. You can complete the entire Colab phase without moving back and forth through this PDF.

AFTER COLAB, RETURN TO THIS PDF

Resume at 3.10B on page 20

Save your notebook and confirm the three core downloads. If you completed the Extra, also keep pl2_extra.zip. Then return to the green RESUME HERE AFTER COLAB banner to start work on the Raspberry Pi.

[Go to 3.10B • Raspberry Pi work • page 20](#)

Before you start

Have the PL1 Raspberry Pi environment, infer_one.py, evaluate_subset.py and test-data files available for the later device work. Training and conversion stay in Colab; the Pi uses LiteRT for inference.

3.1 Colab notebook reference

REFERENCE ONLY — The code and explanations below are already in your Colab notebook. Complete them there once. Return to this PDF at 3.10B, page 20, after the notebook confirms that the Colab work is complete.

Install LiteRT before importing it. Keep the TensorFlow/Keras packages supplied by Colab and record their versions. This notebook was tested in a clean CPU Colab session with Python 3.13.15, TensorFlow 2.20.0, Keras 3.13.2, NumPy 2.1.3 and LiteRT 2.2.0. Do not replace Colab packages just to match these versions.

COLAB · Install the inference runtime

```
%pip install -q "ai-edge-litert==2.2.0"
```

COLAB · Record versions and fixed training settings

```
import sys
import importlib.metadata as metadata
import numpy as np
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

SEED, EPOCHS, BATCH_SIZE = 42, 5, 128
keras.utils.set_random_seed(SEED)
print("Python:", sys.version.split()[0])
print("TensorFlow:", tf.__version__)
print("Keras:", keras.__version__)
print("NumPy:", np.__version__)
print("LiteRT:", metadata.version("ai-edge-litert"))
```

RECORD Save these versions with the notebook. If Colab requests a restart after installation, restart once and rerun from this cell. Never install full TensorFlow on the Pi for this laboratory.

Before continuing

You need the working PL1 LiteRT virtual environment, `infer_one.py`, `evaluate_subset.py` and the two Fashion-MNIST test-data files on the Pi. Use a separate PL2 folder so that the model supplied in PL1 is preserved.

WORKING METHOD Predict the result, execute one block, record the evidence, then explain it. A cell that runs without an exception is only one part of the result.

3.2 Load, split and inspect Fashion-MNIST

The loader returns 60,000 official training images and 10,000 test images. Pixels arrive as uint8 values in 0..255. Convert once to float32 in 0..1, matching the PL1 inference script. Keep the test order unchanged: deployment checks use indices 0..499.

COLAB · Load and normalize

```
(x_all, y_all), (x_test, y_test) = (
    keras.datasets.fashion_mnist.load_data()
)
print("official train:", x_all.shape, x_all.dtype)
print("official test:", x_test.shape, x_test.dtype)
x_all = x_all.astype(np.float32) / 255.0
x_test = x_test.astype(np.float32) / 255.0
assert x_all.shape == (60000, 28, 28)
assert x_test.shape == (10000, 28, 28)
print("normalized range:", float(x_all.min()), float(x_all.max()))
```

Use a fixed shuffle to reserve 6,000 validation images. The remaining 54,000 train every model. This random split is not stratified, so class counts need not be exactly equal. Never create a new split for each architecture.

COLAB · Define the shared training and validation sets

```
order = np.random.default_rng(SEED).permutation(len(x_all))
val_idx, train_idx = order[:6000], order[6000:]
x_train, y_train = x_all[train_idx], y_all[train_idx]
x_val, y_val = x_all[val_idx], y_all[val_idx]
print("train / validation / test:", len(x_train), len(x_val), len(x_test))
print("train class counts:", np.bincount(y_train, minlength=10))
print("validation counts:", np.bincount(y_val, minlength=10))
assert not np.intersect1d(train_idx, val_idx).size
```

Before training, inspect the images behind the numeric labels. The next block defines LABELS in the dataset's class-index order and selects the first training image found for each of the ten classes.

The images appear in a 2×5 grid, with the class index and name above each example. The fixed grayscale range of 0..1 matches the normalized pixels. Use this visual check to spot similar-looking classes and confirm the label mapping; it does not train a model or use test images.

COLAB · Inspect one training example per class

```
LABELS = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot",
]
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for label, ax in enumerate(axes.flat):
    i = np.flatnonzero(y_train == label)[0]
    ax.imshow(x_train[i], cmap="gray", vmin=0, vmax=1)
    ax.set_title(f"{label}: {LABELS[label]}", fontsize=9)
    ax.axis("off")
plt.tight_layout()
plt.show()
```

INTERPRET Which classes look similar? Are the class counts roughly balanced? Why would normalization only during training break the deployment contract? Class balance alone does not establish that a dataset is unbiased.

3.3 Build and train the linear baseline

Flatten changes the shape from 28×28 to 784 without learning parameters. Dense(10) maps pixels directly to class scores; Softmax produces ten scores that sum to one. With integer labels 0..9, sparse categorical cross-entropy is the appropriate loss. Its default expects probabilities, matching the Softmax output.

PREDICT Compute $(784 + 1) \times 10$ parameters before running `summary()`. Multiply by four to estimate Float32 parameter bytes. This excludes activations, runtime memory, metadata and training optimizer state.

COLAB · Define and compile the baseline

```
keras.utils.set_random_seed(SEED)
baseline = keras.Sequential([
    keras.layers.Input(shape=(28, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(10, activation="softmax"),
])
baseline.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"],
)
baseline.summary()
assert baseline.count_params() == 7850
```

Train for five epochs using batches of 128. Validation measures the same reserved images after each epoch without updating weights on those images. An epoch covers all 54,000 training examples; the last batch may be smaller than 128.

COLAB · Train and evaluate on validation data

```
baseline_history = baseline.fit(
    x_train, y_train,
    epochs=EPOCHS, batch_size=BATCH_SIZE,
    validation_data=(x_val, y_val), shuffle=True, verbose=2,
)
baseline_val_loss, baseline_val_acc = baseline.evaluate(
    x_val, y_val, verbose=0,
)
print("baseline validation loss / accuracy:",
      baseline_val_loss, baseline_val_acc)
```

Item	Your result
Parameter count	
Float32 parameter bytes	
Final validation loss	
Final validation accuracy	

INTERPRET The expected parameter count is 7,850 (31,400 parameter bytes). What do the training and validation results suggest about model capacity? Do not inspect test performance yet.

3.4 Build and train the reference MLP

Add a hidden Dense layer with 64 ReLU units. The function below builds a fresh model and a fresh optimizer, resetting the seed each time. The baseline and MLP use the same data, five epochs, batch size, loss and learning rate. Adding this layer changes both capacity and non-linearity.

COLAB · Define the MLP and check its parameter count

```
def build_mlp(hidden_units=64):
    keras.utils.set_random_seed(SEED)
    model = keras.Sequential([
        keras.layers.Input(shape=(28, 28)),
        keras.layers.Flatten(),
        keras.layers.Dense(hidden_units, activation="relu"),
        keras.layers.Dense(10, activation="softmax"),
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss=keras.losses.SparseCategoricalCrossentropy(),
        metrics=["accuracy"],
    )
    return model

mlp = build_mlp(64)
mlp.summary()
assert mlp.count_params() == 50890
```

PREDICT Explain $(784 + 1) \times 64 + (64 + 1) \times 10 = 50,890$ parameters. The parameter storage alone is 203,560 bytes: 203.56 kB or about 198.79 KiB.

COLAB · Train the reference MLP

```
mlp_history = mlp.fit(
    x_train, y_train,
    epochs=EPOCHS, batch_size=BATCH_SIZE,
    validation_data=(x_val, y_val), shuffle=True, verbose=2,
)
mlp_val_loss, mlp_val_acc = mlp.evaluate(x_val, y_val, verbose=0)
print("MLP validation loss / accuracy:", mlp_val_loss, mlp_val_acc)
print("validation gain (percentage points):",
      100 * (mlp_val_acc - baseline_val_acc))
```

Item	Your result
Baseline validation accuracy	
MLP64 validation accuracy	
Accuracy gain in percentage points	
Parameter ratio MLP64 / baseline	

INTERPRET Does the validation improvement justify 43,040 extra parameters? Support the answer with values. Equal epochs do not mean equal runtime, and one seeded run does not prove one architecture is always better.

3.5 Read the curves and change one variable

First examine accuracy and loss over epochs 1..5. Falling training loss with rising validation loss suggests overfitting; a gap alone does not establish it. Do not expect the schematic background curves to match your run exactly.

COLAB · Plot training and validation curves

```
epochs = range(1, EPOCHS + 1)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, metric in zip(axes, ("accuracy", "loss")):
    ax.plot(epochs, mlp_history.history[metric], label="train")
    ax.plot(epochs, mlp_history.history["val_" + metric], label="validation")
    ax.set(xlabel="epoch", ylabel=metric)
    ax.set_xticks(list(epochs))
    ax.grid(alpha=0.3)
    ax.legend()
plt.tight_layout()
plt.show()
```

Run one controlled experiment: change the hidden width from 64 to 32. Keep the split, preprocessing, seed, optimizer, learning rate, epochs and batch size fixed. Write your prediction first. The separate candidate variable preserves the trained 64-unit reference model.

COLAB · Train a 32-unit candidate

```
candidate = build_mlp(32)
candidate_history = candidate.fit(
    x_train, y_train,
    epochs=EPOCHS, batch_size=BATCH_SIZE,
    validation_data=(x_val, y_val), shuffle=True, verbose=2,
)
candidate_val_loss, candidate_val_acc = candidate.evaluate(
    x_val, y_val, verbose=0,
)
print("32-unit parameters:", candidate.count_params())
print("32-unit validation loss / accuracy:",
      candidate_val_loss, candidate_val_acc)
```

Item	Your result
Hypothesis before running	
32-unit parameter count	
32-unit validation loss and accuracy	
Conclusion from 32 vs 64 units	

DECIDE BEFORE TESTING For the optional Extra, complete the Colab steps 4.1–4.7 now, then return to 3.6. Measure memory on the Pi in 4.8–4.9 after 3.12. Otherwise continue to 3.6. The core deployment model remains the original five-epoch MLP64.

3.6 Freeze the experiment and analyse test errors

Record your development conclusion and freeze the settings. Evaluate the two predeclared reference models on all 10,000 test images. These results describe the completed experiment; do not use test errors to tune another version and claim the same test is still independent.

COLAB · Final evaluation and confusion matrix

```
# Freeze the three architectures and training settings before this cell.
# The 64-unit MLP is the fixed deployment reference for this laboratory.
for name, model in [("baseline", baseline), ("MLP64", mlp)]:
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    print(name, "test loss:", test_loss, "test accuracy:", test_acc)

scores = mlp.predict(x_test, verbose=0)
predicted = np.argmax(scores, axis=1)
truth = y_test.astype(int)
confusion = np.zeros((10, 10), dtype=int)
np.add.at(confusion, (truth, predicted), 1)
print("rows=true; columns=predicted; class order:", LABELS)
print(confusion)
assert confusion.sum() == len(y_test)
```

Rows are true classes and columns are predictions. Inspect the two largest off-diagonal entries and the five most confident mistakes. The images help distinguish a plausible visual ambiguity from an unexpected failure. A high Softmax score does not guarantee correctness.

COLAB · Locate confusions and display confident errors

```
off_diagonal = confusion.copy()
np.fill_diagonal(off_diagonal, 0)
largest = np.argsort(off_diagonal.ravel())[::-1][:2]
for pos in largest:
    actual, guessed = np.unravel_index(pos, off_diagonal.shape)
    print(LABELS[actual], "->", LABELS[guessed], off_diagonal[actual, guessed])

error_idx = np.flatnonzero(predicted != truth)
error_scores = scores[error_idx, predicted[error_idx]]
ranked = error_idx[np.argsort(error_scores)::-1][:5]
fig, axes = plt.subplots(1, 5, figsize=(11, 3))
for ax in axes:
    ax.axis("off")
for ax, i in zip(axes, ranked):
    ax.imshow(x_test[i], cmap="gray", vmin=0, vmax=1)
    ax.set_title(f"#{i} true: {LABELS[truth[i]]}\n"
                 f"pred: {LABELS[predicted[i]]}\n"
                 f"score: {scores[i, predicted[i]]:.3f}", fontsize=8)
plt.tight_layout()
plt.show()
```

RECORD Record baseline and MLP test accuracy, the two main class confusions and one confident mistake. Explain why the mistake is plausible or surprising. Any later model change starts a new exploratory experiment; report test reuse explicitly.

3.7 Save and convert the reference MLP

Save the 64-unit MLP in both formats. The .keras file preserves the development model, including training configuration and optimizer state; the .tflite file is the deployment artifact. No quantization is requested here. PL3 will deliberately change numerical representation.

COLAB · Save the Keras model and convert to Float32

```
from pathlib import Path
import hashlib

keras_path = Path("fashion_mnist_mlp.keras")
tflite_path = Path("fashion_mnist_mlp.tflite")
mlp.save(keras_path)
converter = tf.lite.TFLiteConverter.from_keras_model(mlp)
tflite_path.write_bytes(converter.convert())
sha256 = hashlib.sha256(tflite_path.read_bytes()).hexdigest()
print("Keras bytes:", keras_path.stat().st_size)
print("TFLite bytes:", tflite_path.stat().st_size)
print("TFLite SHA-256:", sha256)
```

INTERPRET Why is file size different from parameter bytes? The two formats include different structures and metadata, and the Keras archive can include optimizer state. Do not interpret their size ratio as a measured runtime-memory saving.

3.8 Verify the converted tensor contract

Use the same LiteRT Interpreter API as PL1. A successful conversion only proves a file was generated. Check the input and output tensors before inference; the assertions require Float32 input (1, 28, 28) and output (1, 10). Pixel normalization remains outside the model. Stop and inspect any assertion failure before transfer.

COLAB · Inspect shape and dtype

```
from ai_edge_litert.interpreter import Interpreter

interpreter = Interpreter(model_path=str(tflite_path))
interpreter.allocate_tensors()
input_info = interpreter.get_input_details()[0]
output_info = interpreter.get_output_details()[0]
print("input:", input_info["shape"], input_info["dtype"])
print("output:", output_info["shape"], output_info["dtype"])
assert tuple(input_info["shape"]) == (1, 28, 28)
assert tuple(output_info["shape"]) == (1, 10)
assert input_info["dtype"] == np.float32
assert output_info["dtype"] == np.float32
```

Item	Your result
Keras artifact bytes	
TFLite artifact bytes	
Input shape and dtype	
Output shape and dtype	
SHA-256 recorded with the artifact	

3.9 Compare Keras and LiteRT on the same inputs

Evaluate both runtimes on exactly the first 500 test images in their original order. This checks more than a single prediction. Report accuracy, prediction disagreement and the largest absolute score difference; equal accuracy can hide different errors.

COLAB · Compare all 500 outputs

```
limit = 500 # Same ordered test subset as evaluate_subset.py in PL1.
keras_scores = mlp.predict(x_test[:limit], verbose=0)
lite_scores = np.empty_like(keras_scores)
for i in range(limit):
    interpreter.set_tensor(input_info["index"], x_test[i:i + 1])
    interpreter.invoke()
    lite_scores[i] = interpreter.get_tensor(output_info["index"])[0]

keras_pred = keras_scores.argmax(axis=1)
lite_pred = lite_scores.argmax(axis=1)
keras_acc = float(np.mean(keras_pred == y_test[:limit]))
lite_acc = float(np.mean(lite_pred == y_test[:limit]))
max_diff = float(np.max(np.abs(keras_scores - lite_scores)))
disagreements = np.flatnonzero(keras_pred != lite_pred)
print("Keras / LiteRT first-500 accuracy:", keras_acc, lite_acc)
print("max absolute score difference:", max_diff)
print("disagreement indices:", disagreements.tolist())
```

INTERPRET Small Float32 score differences can be normal. Investigate class disagreements, especially near ties, before transfer; a single disagreement does not by itself prove conversion failed. Check shape, dtype, normalization and output order first. Do not tune the model from this deployment check.

Save the artifact fingerprint and reference results so that later Pi evidence can be linked to the exact model. This JSON is a human-readable record, not another model file.

COLAB · Save the deployment reference

```
import json

reference = {
    "test_subset": "official Fashion-MNIST test indices 0..499",
    "sha256": sha256,
    "seed": SEED, "epochs": EPOCHS, "batch_size": BATCH_SIZE,
    "tensorflow": tf.__version__, "keras": keras.__version__,
    "numpy": np.__version__, "litert": metadata.version("ai-edge-litert"),
    "keras_accuracy": keras_acc, "litert_accuracy": lite_acc,
    "max_score_difference": max_diff,
    "disagreement_indices": disagreements.tolist(),
}
Path("pl2_reference.json").write_text(json.dumps(reference, indent=2))
```

Item	Your result
Keras first-500 accuracy	
LiteRT first-500 accuracy	
Number of prediction disagreements	
Maximum absolute score difference	

3.10A Complete the Colab downloads

COLAB NOTEBOOK REFERENCE — This is the final Colab step. If you have already downloaded the files in the notebook, go directly to 3.10B on the next page.

Download the deployment model, development model and JSON to your computer, and save the notebook with outputs. Confirm all three files are present before continuing. If a download is missing, rerun that line separately or use Files > right-click the file > Download.

COLAB · Download the three artifacts

```
from google.colab import files
files.download("fashion_mnist_mlp.tflite")
files.download("fashion_mnist_mlp.keras")
files.download("pl2_reference.json")
```

Before leaving Colab

Confirm that your computer contains `fashion_mnist_mlp.tflite`, `fashion_mnist_mlp.keras` and `pl2_reference.json`. Keep your notebook saved with outputs and observations. If you completed the Extra, also confirm that `pl2_extra.zip` is present.

The `.tflite` file goes to the Pi. Keep the `.keras` model and notebook as development evidence, and use `pl2_reference.json` to compare the model hash and the first-500 accuracy. The Extra ZIP contains the five models and memory helpers for 4.8.

Colab complete? Continue at 3.10B · page 20

3.10B Resume on the Raspberry Pi

RESUME HERE AFTER COLAB

Your next task is on the Raspberry Pi

Start here after saving your notebook and receiving the downloads. Follow 3.10B–3.12 in this PDF. Run RASPBERRY PI blocks in the Pi terminal and LAPTOP blocks on your computer. Keep the Colab results available for comparison.

On the Pi, reuse the PL1 virtual environment, loaders and evaluator. Keep the test files and script contents unchanged so the newly trained model is the intended variable. If a required file is missing, recover it from PL1 before continuing.

RASPBERRY PI · Prepare the PL2 workspace

```
mkdir -p ~/ai-embedded/pl2/model ~/ai-embedded/pl2/data
cp ~/ai-embedded/pl1/data/t10k-*.gz ~/ai-embedded/pl2/data/
cp ~/ai-embedded/pl1/infer_one.py ~/ai-embedded/pl2/
cp ~/ai-embedded/pl1/evaluate_subset.py ~/ai-embedded/pl2/
cd ~/ai-embedded/pl2
source ~/ai-embedded/pl1/.venv/bin/activate
python -m pip show ai-edge-litert
hostname -I
```

3.11 Transfer and prove artifact identity

On your laptop, open a terminal in the download folder. Replace YOUR_PI_IP with the Pi IPv4 address. The command works in common Linux, macOS and Windows OpenSSH terminals. Use the same network as the Pi and the lab account iase; use the password supplied in class.

LAPTOP · Transfer only the inference model

```
scp fashion_mnist_mlp.tflite iase@YOUR_PI_IP:~/ai-embedded/pl2/model/
```

Back on the Pi, compare the complete SHA-256 value with the notebook or pl2_reference.json. Record the byte count rather than relying only on a rounded human-readable size.

RASPBERRY PI · Verify the received artifact

```
cd ~/ai-embedded/pl2
sha256sum model/fashion_mnist_mlp.tflite
wc -c model/fashion_mnist_mlp.tflite
```

DECISION GATE If the hashes differ, stop and transfer the intended file again. Matching hashes establish file identity; they do not establish correct preprocessing or runtime behaviour.

3.12 Run the model and collect deployment evidence

In the activated PL1 environment, run from ~/ai-embedded/pl2. The reused scripts read the model and data using relative paths. The argument to infer_one.py is a test-image index, not a class label. Predictions may differ from PL1 because you trained a new model.

RASPBERRY PI • Run inference and the fixed subset evaluation

```
python infer_one.py 25
python infer_one.py 0
python infer_one.py 1
python infer_one.py 42
python evaluate_subset.py
```

The evaluator uses test indices 0..499, exactly as the Colab comparison. Compare Pi/LiteRT first with Colab/LiteRT, then with Keras. Do not compare this 500-image accuracy directly with the complete 10,000-image test accuracy as if they described identical samples.

Item	Your result
Pi hostname / IPv4 address	
Pi LiteRT version	
SHA-256 identical?	
Keras first-500 accuracy	
Colab/LiteRT first-500 accuracy	
Pi/LiteRT first-500 accuracy	
Absolute accuracy difference in percentage points	

3.13 Diagnose a mismatch systematically

Symptom	First check
Import error on the Pi	Activate the PL1 environment; verify the LiteRT package.
File not found	Check the current folder, data files and exact model filename.
Shape or dtype failure	Inspect the contract: batch 1, 28×28, Float32.
Identical hash but different results	Check normalization, sample order, scripts and runtime versions.
Small score changes near a tie	Compare scores and top classes; do not assume bitwise identity.

ENGINEERING CONCLUSION A deployment claim requires the artifact identity, matching data and tensor contracts, working inference and measured target results. Accuracy alone cannot establish that every prediction agrees between devices.

[Extra completed in Colab? Go directly to 4.8 • Pi memory • page 30](#)

4

Extra

Training time, width, depth and memory on the Raspberry Pi

COLAB NOTEBOOK REFERENCE · 4.1-4.7 These steps are already included in the notebook before 3.6. If you completed them in Colab and have finished 3.12 on the Pi, skip directly to 4.8 on page 30. Do not train the same models again.

Five epochs are a starting point. In this Extra you will train for longer, compare wider and deeper networks, and measure the deployed candidates on the Raspberry Pi. The aim is to relate validation quality to storage and RAM cost. More epochs, neurons or layers do not guarantee higher accuracy.

4.1 Give the same network more time

Start a fresh 64-unit MLP and train for 30 epochs. The split, normalization, seed, optimizer, learning rate and batch size remain fixed. EXTRA_EPOCHS is separate from EPOCHS, so the core exercise stays at five epochs. Each output line reports loss and accuracy for one epoch.

ModelCheckpoint is a callback: Keras calls it during training. Here it saves the complete model whenever validation loss improves. Training still runs for all 30 epochs; the in-memory extra64 ends at epoch 30, while the saved file may contain an earlier, better epoch.

PREDICT Will validation accuracy keep rising? Will its loss keep falling? Write a prediction before running. This experiment adds training work without adding parameters to the inference model.

COLAB · Train a fresh 64-unit MLP for 30 epochs

```
EXTRA_EPOCHS = 30
extra64 = build_mlp(64) # Fresh weights and optimizer; same seed.
save64 = keras.callbacks.ModelCheckpoint(
    "extra_mlp64_best.keras", monitor="val_loss", mode="min",
    save_best_only=True, verbose=0,
)
extra64_history = extra64.fit(
    x_train, y_train,
    epochs=EXTRA_EPOCHS, batch_size=BATCH_SIZE,
    validation_data=(x_val, y_val), shuffle=True, verbose=2,
    callbacks=[save64],
)
print("Extra model parameters:", extra64.count_params())
```

KEEP THE COMPARISON CLEAN All Extra decisions use validation data. If you have already inspected the test results in 3.6, treat this as further exploration and disclose test reuse; do not claim a new independent final result.

4.2 Watch loss and accuracy across epochs

The next block plots the complete learning history. The dotted line marks the original five-epoch budget. The green line marks the epoch with the lowest validation loss, which is the criterion used to save the checkpoint. The printed values compare epoch 5, epoch 30 and that selected epoch within the same training run.

COLAB · Plot the curves and locate the best validation loss

```
h64 = extra64_history.history
extra_epochs = np.arange(1, len(h64["loss"]) + 1)
best64_epoch = int(np.argmin(h64["val_loss"])) + 1
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, metric in zip(axes, ("loss", "accuracy")):
    ax.plot(extra_epochs, h64[metric], label="train")
    ax.plot(extra_epochs, h64["val_" + metric], label="validation")
    ax.axvline(EPOCHS, color="gray", ls=":", label="5 epochs")
    ax.axvline(best64_epoch, color="green", ls="--",
               label="lowest validation loss")
    ax.set(xlabel="epoch", ylabel=metric)
    ax.grid(alpha=0.3)
    ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

for epoch in dict.fromkeys([EPOCHS, EXTRA_EPOCHS, best64_epoch]):
    i = epoch - 1
    print(f"epoch {epoch:2d}: train loss={h64['loss'][i]:.4f}, "
          f"val loss={h64['val_loss'][i]:.4f}, "
          f"val accuracy={100 * h64['val_accuracy'][i]:.2f}%")
```

Look for a trend rather than a perfectly smooth descent: training uses mini-batches, so a loss value can rise between epochs. Training loss is averaged while weights are changing; validation loss is measured after the epoch. The two curves need not start in the order you expect.

INTERPRET Accuracy counts correct labels; loss also responds to the scores assigned to those labels. Accuracy can change little while loss worsens. If training loss keeps falling but validation loss rises, more fitting is no longer helping according to that validation criterion.

Item	Your result
Validation loss and accuracy at epoch 5	
Validation loss and accuracy at epoch 30	
Epoch with the lowest validation loss	
Evidence that further training helped or stopped helping	

4.3 Does a wider hidden layer help?

Now compare 64 and 128 hidden units with the same 30-epoch budget. The only changed setting is hidden width; each model starts fresh with the same seed. Its number and arrangement of initial weights necessarily change with its shape. Keep the data split, optimizer, learning rate and batch size fixed.

PREDICT The wider MLP has $(784 + 1) \times 128 + (128 + 1) \times 10 = 101,770$ parameters. Will roughly doubling the parameters bring a useful improvement? State a hypothesis before running.

COLAB · Train the 128-unit candidate for the same 30 epochs

```
extra128 = build_mlp(128)
save128 = keras.callbacks.ModelCheckpoint(
    "extra_mlp128_best.keras", monitor="val_loss", mode="min",
    save_best_only=True, verbose=0,
)
extra128_history = extra128.fit(
    x_train, y_train,
    epochs=EXTRA_EPOCHS, batch_size=BATCH_SIZE,
    validation_data=(x_val, y_val), shuffle=True, verbose=2,
    callbacks=[save128],
)
print("Wider model parameters:", extra128.count_params())
assert extra128.count_params() == 101770
```

Overlay the validation curves to see whether the wider model learns faster, reaches a better result or starts to overfit sooner. A comparison of 128 units at 30 epochs against 64 units at five epochs changes two factors; use the 64-unit, 30-epoch run to discuss width.

COLAB · Compare validation curves at equal training budgets

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for width, history in [(64, extra64_history), (128, extra128_history)]:
    h = history.history
    steps = np.arange(1, len(h["loss"]) + 1)
    for ax, metric in zip(axes, ("val_loss", "val_accuracy")):
        ax.plot(steps, h[metric], label=f"{width} units")
        ax.set(xlabel="epoch", ylabel=metric)
        ax.grid(alpha=0.3)
        ax.legend()
plt.tight_layout()
plt.show()
```

READ THE EVIDENCE Which width has the lower validation loss? Does it also have the higher validation accuracy? Look across the full curves before deciding. A larger network can fit training data better without improving unseen-data performance.

4.4 Keep the best epoch and defend a choice

The final weights and the best saved weights can differ. Reload both saved checkpoints and evaluate them on the same validation set. The selection rule was fixed before training: lowest validation loss, not highest accuracy. The best epoch under one metric need not be best under the other.

COLAB · Reload the saved models and compare validation evidence

```
best64 = keras.models.load_model("extra_mlp64_best.keras")
best128 = keras.models.load_model("extra_mlp128_best.keras")
best128_epoch = int(np.argmax(extra128_history.history["val_loss"])) + 1
extra_models = [
    ("64 units / 5 epochs", mlp, EPOCHS),
    ("64 units / last epoch", extra64, EXTRA_EPOCHS),
    ("64 units / lowest val loss", best64, best64_epoch),
    ("128 units / lowest val loss", best128, best128_epoch),
]
extra_results = []
for name, model, epoch in extra_models:
    loss, acc = model.evaluate(x_val, y_val, verbose=0)
    extra_results.append((name, epoch, model.count_params(), loss, acc))
print(f"{name}: epoch={epoch}, params={model.count_params()}, "
      f"val loss={loss:.4f}, val accuracy={100 * acc:.2f}%, "
      f"gain={100 * (acc - mlp_val_acc):+.2f} pp")
```

The gain is measured in percentage points relative to the original five-epoch MLP. It is development evidence from the validation set used to select checkpoints, not an independent test estimate. One seeded run does not establish that a small difference will repeat.

Item	Your result
Preferred model and selected epoch	
Validation accuracy gain over the five-epoch reference	
Parameter count and reason for accepting that cost	
One limitation of your conclusion	

To keep the Extra models, download `extra_mlp64_best.keras` and `extra_mlp128_best.keras` from the Colab Files panel. Saving the notebook alone does not save these model files to your computer.

EXPLAIN Did more epochs help without increasing parameter count? Did 128 units justify their extra parameters? Why might the lowest-loss checkpoint differ from the highest-accuracy epoch? Support your answers with the plotted curves and recorded values.

CONTINUE THE EXTRA Continue at 4.5 to compare depth and width, then export the candidates in 4.7. Keep the original mlp variable for the core deployment. The Extra uses separate model variables and files.

4.5 Change width and depth separately

Width means the number of neurons in each hidden layer. Depth means the number of hidden Dense layers; the ten-class output layer is always present and is not counted in depth here. Compare 32, 64 and 128 neurons at depth 1, then compare depths 1, 2 and 3 at width 64. Every run uses the same 30-epoch budget and selects its checkpoint by lowest validation loss.

COLAB · Build an MLP with configurable width and depth

```
def build_extra_mlp(width, depth):
    keras.utils.set_random_seed(SEED)
    layers = [keras.layers.Input(shape=(28, 28)), keras.layers.Flatten()]
    for _ in range(depth):
        layers.append(keras.layers.Dense(width, activation="relu"))
    layers.append(keras.layers.Dense(10, activation="softmax"))
    model = keras.Sequential(layers)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss=keras.losses.SparseCategoricalCrossentropy(),
        metrics=["accuracy"],
    )
    return model

# Depth counts hidden Dense layers; the output layer is always present.
arch_models = {"d1_w64": best64, "d1_w128": best128}
arch_histories = {"d1_w64": extra64_history, "d1_w128": extra128_history}
arch_settings = {"d1_w64": (1, 64), "d1_w128": (1, 128)}
```

Reuse the 64- and 128-unit checkpoints already trained in 4.1–4.4. The next block trains only the three missing candidates: one layer of 32, two layers of 64 and three layers of 64. Each starts from new weights and a new optimizer. The assertion checks the parameter count before training.

PREDICT Will adding a second hidden layer help more than doubling width? Will a third layer always improve accuracy? Make these predictions before inspecting the results.

COLAB · Train the remaining controlled candidates

```
for depth, width in [(1, 32), (2, 64), (3, 64)]:
    name = f"d{depth}_w{width}"
    model = build_extra_mlp(width, depth)
    expected = 785 * width + (depth - 1) * (width + 1) * width
    expected += (width + 1) * 10
    assert model.count_params() == expected
    print(name, "parameters:", model.count_params())
    checkpoint = keras.callbacks.ModelCheckpoint(
        name + "_best.keras", monitor="val_loss", mode="min",
        save_best_only=True, verbose=0,
    )
    history = model.fit(
        x_train, y_train, epochs=EXTRA_EPOCHS, batch_size=BATCH_SIZE,
        validation_data=(x_val, y_val), shuffle=True, verbose=2,
        callbacks=[checkpoint],
    )
    arch_models[name] = keras.models.load_model(name + "_best.keras")
    arch_histories[name] = history
    arch_settings[name] = (depth, width)
```

4.6 Read the architecture comparisons

The top row below varies width at a fixed depth. The bottom row varies depth at a fixed width. Both show validation loss and accuracy throughout training. The final printed metrics come from each saved checkpoint, not automatically from epoch 30.

COLAB · Plot width and depth as two separate experiments

```
fig, axes = plt.subplots(2, 2, figsize=(11, 7))
families = [
    ("Width at depth 1", ["d1_w32", "d1_w64", "d1_w128"]),
    ("Depth at width 64", ["d1_w64", "d2_w64", "d3_w64"]),
]
for row, (title, names) in enumerate(families):
    for name in names:
        h = arch_histories[name].history
        steps = np.arange(1, len(h["loss"]) + 1)
        for ax, metric in zip(axes[row], ("val_loss", "val_accuracy")):
            ax.plot(steps, h[metric], label=name)
            ax.set(title=title, xlabel="epoch", ylabel=metric)
            ax.grid(alpha=0.3)
            ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

for name in ["d1_w32", "d1_w64", "d1_w128", "d2_w64", "d3_w64"]:
    loss, acc = arch_models[name].evaluate(x_val, y_val, verbose=0)
    print(name, "params:", arch_models[name].count_params(),
          "val loss:", round(loss, 4), "val accuracy:", round(acc, 4))
```

Candidate	Architecture and expected parameter count
d1_w32	1 hidden layer × 32 neurons; 25,450 parameters
d1_w64	1 hidden layer × 64 neurons; 50,890 parameters
d1_w128	1 hidden layer × 128 neurons; 101,770 parameters
d2_w64	2 hidden layers × 64 neurons; 55,050 parameters
d3_w64	3 hidden layers × 64 neurons; 59,210 parameters

INTERPRET Adding a 64-to-64 hidden layer introduces 4,160 parameters. Increasing the first layer from 64 to 128 is much more expensive because it receives all 784 pixels. More depth changes the function and optimization problem; it does not guarantee a better result.

RECORD Record validation loss, accuracy, selected epoch and parameters for every candidate. Choose one width comparison and one depth comparison. State what changed, what you observed, and whether the gain seems worth its cost. Do not tune these choices on the test set.

4.7 Export the candidates for the same device test

Convert the five selected models to Float32 LiteRT. The block checks the tensor contract and compares the first 100 validation images between Keras and LiteRT. A manifest records model identity, architecture and validation results. One normalized validation image is saved for a consistent memory workload; the Pi will not load the entire dataset during these measurements.

COLAB · Convert all candidates and record their identities

```
from pathlib import Path
import hashlib
import json
from ai_edge_litert.interpreter import Interpreter

bundle = Path("pl2_extra")
(bundle / "models").mkdir(parents=True, exist_ok=True)
np.save(bundle / "memory_sample.npy", x_val[:1])
manifest = {"sample": "first image in the fixed validation split",
            "seed": SEED, "epochs": EXTRA_EPOCHS, "models": []}
for name in ["d1_w32", "d1_w64", "d1_w128", "d2_w64", "d3_w64"]:
    selected = arch_models[name]
    converter = tf.lite.TFLiteConverter.from_keras_model(selected)
    path = bundle / "models" / (name + ".tflite")
    path.write_bytes(converter.convert())
    runtime = Interpreter(model_path=str(path), num_threads=1)
    runtime.allocate_tensors()
    inp, out = runtime.get_input_details()[0], runtime.get_output_details()[0]
    assert tuple(inp["shape"]) == (1, 28, 28) and inp["dtype"] == np.float32
    assert tuple(out["shape"]) == (1, 10) and out["dtype"] == np.float32
    converted = []
    for i in range(100):
        runtime.set_tensor(inp["index"], x_val[i:i + 1])
        runtime.invoke()
        converted.append(runtime.get_tensor(out["index"])[0])
    converted = np.asarray(converted)
    original = selected.predict(x_val[:100], verbose=0)
    max_diff = float(np.max(np.abs(original - converted)))
    disagreements = int(np.sum(original.argmax(1) != converted.argmax(1)))
    print(name, "bytes:", path.stat().st_size, "score diff:", max_diff,
          "class disagreements:", disagreements)
    assert np.isfinite(converted).all()
    assert max_diff < 1e-4, "Inspect conversion before transfer."
    loss, acc = selected.evaluate(x_val, y_val, verbose=0)
    depth, width = arch_settings[name]
    manifest["models"].append({
        "name": name, "file": "models/" + path.name,
        "depth": depth, "width": width, "parameters": selected.count_params(),
        "file_bytes": path.stat().st_size,
        "sha256": hashlib.sha256(path.read_bytes()).hexdigest(),
        "validation_loss": float(loss), "validation_accuracy": float(acc),
        "best_epoch": int(np.argmin(arch_histories[name].history["val_loss"])) + 1,
        "sample_scores": converted[0].astype(float).tolist(),
        "sample_class": int(converted[0].argmax()),
        "conversion_max_diff": max_diff, "conversion_disagreements": disagreements,
    })
(bundle / "manifest.json").write_text(json.dumps(manifest, indent=2))
```

CHECK BEFORE TRANSFER Inspect every conversion difference and class disagreement. The 1e-4 score check catches substantial numerical differences; near-tie class changes still need interpretation. This is a conversion check using validation data, not a new independent accuracy evaluation.

4.7 Package the models and memory helpers

Use the official v6 notebook: its Prepare the memory helpers cell creates `measure_memory.py` and `compare_memory.py` automatically. Run that cell before the ZIP-packaging block below. Keep the helper code unchanged so that every candidate is measured consistently.

Bundle component	Purpose
<code>models/*.tflite</code>	Five Float32 deployment artifacts.
<code>memory_sample.npy</code>	One normalized input shared by all models.
<code>manifest.json</code>	Hashes, architectures and Colab reference results.
<code>measure_memory.py</code>	Loads one model, runs inference and samples its process RAM.
<code>compare_memory.py</code>	Starts fresh processes, repeats measurements and writes a summary.

COLAB · Build one ZIP containing the complete experiment

```
import shutil
import zipfile
for helper in ["measure_memory.py", "compare_memory.py"]:
    shutil.copy2(helper, bundle / helper)
included = ["measure_memory.py", "compare_memory.py",
            "memory_sample.npy", "manifest.json"]
included += [item["file"] for item in manifest["models"]]
with zipfile.ZipFile("pl2_extra.zip", "w", zipfile.ZIP_DEFLATED) as archive:
    for name in included:
        archive.write(bundle / name, arcname="pl2_extra/" + name)
print("Created pl2_extra.zip with models, sample, manifest and helpers.")
```

COLAB · Download the Extra bundle

```
from google.colab import files
files.download("pl2_extra.zip")
```

Save `pl2_extra.zip` on your computer and confirm that the file is present. If needed, rerun only this download cell or use Files > right-click `pl2_extra.zip` > Download. Keep the notebook with its plots and observations separately.

CONTINUE IN THE COLAB NOTEBOOK After the Extra downloads, continue at 3.6 in the notebook and complete its remaining Colab cells. Return to this PDF at 3.10B, page 20. After core deployment, use 4.8–4.9 for the Extra memory measurements on the Pi.

4.8 Measure storage and RAM on the Raspberry Pi

RESUME THE EXTRA HERE — Use the `pl2_extra.zip` you already generated and downloaded in Colab. Complete core deployment in 3.12 first, then follow 4.8–4.9 on the Pi.

A model's storage cost and its running process's RAM use answer different questions. For each architecture, collect the following three quantities. All candidates must run on the same Pi, in the same PL1 LiteRT environment, with one inference thread and the same input.

Quantity	What it tells you
Float32 parameter bytes	$4 \times$ parameter count. This is a theoretical weight-and-bias estimate.
TFLite file bytes	Actual artifact size on storage. It includes the serialized graph and metadata.
Process RSS	Resident RAM occupied by the Python process, including NumPy, LiteRT, model pages and tensors.

From your laptop's download folder, transfer the ZIP to the PL2 folder you created in 3.10B. Replace `YOUR_PI_IP` with the Pi address. The two supplied scripts require Linux and LiteRT, not TensorFlow.

LAPTOP · Transfer the Extra bundle

```
scp pl2_extra.zip iase@YOUR_PI_IP:~/ai-embedded/pl2/
```

On the Pi, activate the existing environment and extract the ZIP. Run the comparison script from the extracted folder. It validates each model hash, invokes each model 100 times on the saved image and repeats the experiment three times in new processes. This avoids keeping earlier interpreters in RAM.

RASPBERRY PI · Run the memory comparison

```
cd ~/ai-embedded/pl2
source ~/ai-embedded/pl1/.venv/bin/activate
python -m zipfile -e pl2_extra.zip .
cd pl2_extra
python compare_memory.py
cat memory_summary.csv
```

WHAT THE HELPER MEASURES `measure_memory.py` reads `Rss` from `/proc/self/smaps_rollup` after importing the runtime, after tensor allocation and after inference. `compare_memory.py` reports the median and range of the three final RSS snapshots. This is not a measurement of the maximum temporary allocation or peak RAM.

4.9 Relate validation quality to measured memory

Open `memory_summary.csv` on the Pi or copy it to your computer. It contains one row per model with the architecture, parameter bytes, file bytes, validation accuracy and measured RSS. `memory_raw.json` retains every repetition, runtime versions, hardware architecture, hashes and single-image output comparisons.

Summary field	How to interpret it
<code>tf_lite_bytes / 1024</code>	Storage size in KiB; compare it with <code>parameter_bytes_fp32 / 1024</code> .
<code>rss_median_kib / 1024</code>	Median resident process memory in MiB.
<code>rss_min_kib</code> and <code>rss_max_kib</code>	Observed variation across three fresh processes.
<code>rss_change_median_kib</code>	Change relative to the imported-runtime baseline; not exact model-only RAM.
<code>sample_class_agrees_all</code>	Whether the saved input gets the same top class as Colab/LiteRT.
<code>sample_score_max_diff</code>	Largest score difference on that one input across repetitions.

RSS includes shared library and model-file pages. The difference between two snapshots also depends on allocation and runtime behaviour, so it can be noisy and need not increase in perfect parameter-count order. The models are small compared with the Python/LiteRT process overhead. Overlapping ranges are a valid result; do not turn tiny differences into a firm ranking. These snapshots do not include memory of the whole board.

INTERPRET At depth 1, how do width, file size and RSS change? At width 64, how do depth and those costs change? Does the candidate with the best validation accuracy justify the measured memory cost? Use 1 KiB = 1024 bytes and 1 MiB = 1024 KiB consistently.

Item	Your result
Preferred candidate and validation accuracy	
TFLite size in KiB and process RSS median in MiB	
RSS range and whether the comparison is conclusive	
One limitation and one follow-up measurement	

RETAIN THE EVIDENCE Keep `memory_summary.csv` and `memory_raw.json` with the manifest and notebook. A matching prediction on one input checks this workload only; it does not establish equal accuracy on a dataset. If predictions disagree, check versions and tensor contracts before drawing conclusions about memory.

5

PL2 checkpoint

Evidence to retain and questions to explain before leaving.

Checkpoint	Evidence to retain
Data and method	Split sizes, versions, seed, training settings and class observations.
Controlled comparison	Baseline/MLP parameter counts, validation metrics and final test metrics.
One-variable experiment	Hypothesis, 32-vs-64 validation results and reasoned conclusion.
Generalisation and errors	Learning curves, confusion matrix and confident error examples.
Conversion	Tensor contract, first-500 comparisons and deployment reference JSON.
Target execution	Matching SHA-256, Pi runtime version, inference and subset results.

Explain the decisions

- Why do training and inference require different work and software stacks?
- How are 7,850 and 50,890 parameters obtained, and what is excluded from parameter-memory estimates?
- Why do we control the split and training budget, and why is validation different from test?
- What changed in the 32-unit experiment, and what does one run allow you to conclude?
- Why can a confident prediction still be wrong?
- Why are conversion success, identical hashes and successful Pi inference different pieces of evidence?
- Why must comparisons use the same model, preprocessing and sample indices?

SUBMISSION CHECK Keep the notebook with outputs, .keras model, .tflite artifact, pl2_reference.json and Pi terminal evidence together. Add a short conclusion citing the main values and one limitation. Use the course submission channel indicated by the lecturer.

LOOK AHEAD Record one unresolved question for PL3 about quantization, quality or system cost. Do not present a hypothesized hardware benefit as a measurement.

Official references

Fashion-MNIST dataset and labels · <https://github.com/zalando-research/fashion-mnist>

Keras model training API · https://keras.io/api/models/model_training_apis/

Keras ModelCheckpoint · https://keras.io/api/callbacks/model_checkpoint/

TensorFlow Lite conversion · https://www.tensorflow.org/api_docs/python/tf/lite/TFLiteConverter

LiteRT Python migration · <https://ai.google.dev/edge/litert/migration>

Google Colab FAQ · <https://research.google.com/colaboratory/faq.html>

Linux process memory · <https://www.kernel.org/doc/html/latest/filesystems/proc.html>